Georgia State University
CSC 4210/6210 Computer Architecture
Spring Semester 2026

Processor Design Semester Project - Task3:

Release Date: 03-29-2026
Due Date: 04-08-2026

Memory Hierarchy Simulation (SSD → DRAM → Cache)

You are designing the memory subsystem of a processor datapath for Samsung semiconductors. In real systems, instructions must travel through a hierarchy (SSD → DRAM → Cache → CPU) without bypassing intermediate levels. This assignment focuses on modeling the data movement across memory hierarchy.

Section 1 — Memory System Design

1. Implement the following components:
    a. SSD (largest storage)
    b. DRAM (intermediate storage)
    c. Cache hierarchy (L3, L2, L1)
2. The system must enforce:
    a. Data flow: SSD → DRAM → L3 → L2 → L1 → CPU
    b. Direct access or bypassing is not allowed
3. All data must be treated as 32-bit instructions

Section 2 — Parameterization

1. Allow configurable sizes for each level:
    a. SSD size
    b. DRAM size
    c. Cache sizes (L3, L2, L1)
2. Sizes must be defined in terms of number of instructions
3. The hierarchy must be followed: SSD > DRAM > L3 > L2 > L1

Section 3 — Data Movement & Clock

1. Simulate the system using a clock-driven model
2. At each clock cycle:
    a. Data may move between levels
    b. Transfers may take multiple cycles (student-defined latency)
3. Optionally:
    a. Limit number of instructions transferred per cycle (bandwidth)

Section 4 — Read / Write Operations

1. Support:
    a. Read operation (fetch instruction)
    b. Write operation (write-back to lower levels)

Section 5 — Cache Behavior

1. When cache is full:
    a. The system must decide which data to evict
2. (Optional – Bonus)
    a. Implement a cache replacement policy
        i. LRU
        ii. FIFO
        iii. Random

Section 6 — Data Movement Rules

1. Data must move hierarchically:
    a. SSD → DRAM → L3 → L2 → L1
2. Data cannot skip any level
3. Movement must be consistent with access requests

Program Output:

1. Memory hierarchy configuration
2. Instruction access trace
3. Movement of data across levels
4. Cache hits / misses (if implemented)
5. Final state of each memory level

Deliverables:

1. Source code (Python preferred)
2. README with instructions
3. Submit code to github
4. 1–2 minute demo video explaining code and displaying input/output
    a. Either upload it to iCollege or storage of your choice with necessary viewing permissions
5. Can video recordings be uploaded to youtube? Yes or No